

Sistemas Operativos

3º Semestre del Grado en Ingeniería Informática

Práctica 4: Procesos. Señales

Alejandro Valero, Rubén Gran, Pablo Ibáñez

Area de Arquitectura y Tecnología de Computadores

Departamento de Informática e Ingeniería de Sistemas

Universidad de Zaragoza

OBJETIVOS DE LA SESIÓN DE PRÁCTICAS

- Procesos y señales: interacción llamadas al sistema `fork()`, `exec()` y señales.
- Uso de las llamadas al sistema `signal()`

EJERCICIOS PREVIOS A LA SESIÓN

1. Describe el comportamiento y los parámetros de las siguientes llamadas al sistema:

- `kill()`
- `signal()`
- `pause()`
- `alarm()`

2. Escribe un programa en C que se envíe a sí mismo un número de señal que le pasaremos como parámetro.

```
$hendrix> ./sendsignalto me 9
```

Se enviará a sí mismo la señal 9 (SIGKILL)

3. ¿Cuáles son los posibles comportamientos que un proceso puede programar ante la mayoría de las señales?
4. En el programa anterior, cambia el comportamiento por defecto a cualquiera de los otros posibles comportamientos de la señal.
5. ¿Qué le ocurre a un comportamiento programado para una señal ante las llamadas al sistema: `fork()`, `exec()` y `signal()`?
6. Lee el enunciado de la práctica
7. Escribe en papel un primer diseño de la función `duerme` descrita en el enunciado

ENUNCIADO DE LA PRÁCTICA

Con la ayuda del sistema de señales de UNIX, implementar la función *duerme(segundos)*. Esta función mantendrá bloqueado al proceso que la llame durante el número de segundos pasado como parámetro.

Utilizar dicha función para realizar un programa (ej91) que lance un mensaje a pantalla cada X segundos durante Y segundos. Para ello, ej91 ejecutará un `fork()` y el proceso hijo lanzará mensajes indefinidamente cada X segundos. Pasados Y segundos, el proceso padre suspenderá la ejecución del proceso hijo (SIGTSTP) durante Y segundos. Pasados estos últimos Y segundos, se retomará la ejecución del proceso hijo (SIGCONT), y pasados otros Y segundos el proceso padre matará al proceso hijo y finalizará. X e Y se pasarán a ej91 como parámetros:

Ejemplo: `./ej91 2 7`

Modificar ej1001.c de forma que el proceso padre quede protegido contra las señales SIGINT y SIGQUIT y los procesos hijos no.

EJERCICIOS POSTERIORES A LA SESIÓN

1. Supongamos el siguiente programa:

```
#include <signal.h>

main(){

    signal(SIGALRM, SIG_DFL);

    alarm(5);

    pause();

}
```

Analiza el código e indica qué ocurrirá al ejecutarlo.

Escribe, compila y ejecuta el programa. Verifica si tu análisis ha sido correcto.

2. Si en el código anterior cambiamos el comportamiento por defecto a ignorar, ¿qué ocurrirá?
3. En el programa ej91, una vez que el proceso padre ha suspendido la ejecución del hijo, ¿en qué estado se encuentra el hijo?

Compruébalo ejecutando el comando `ps -l`.